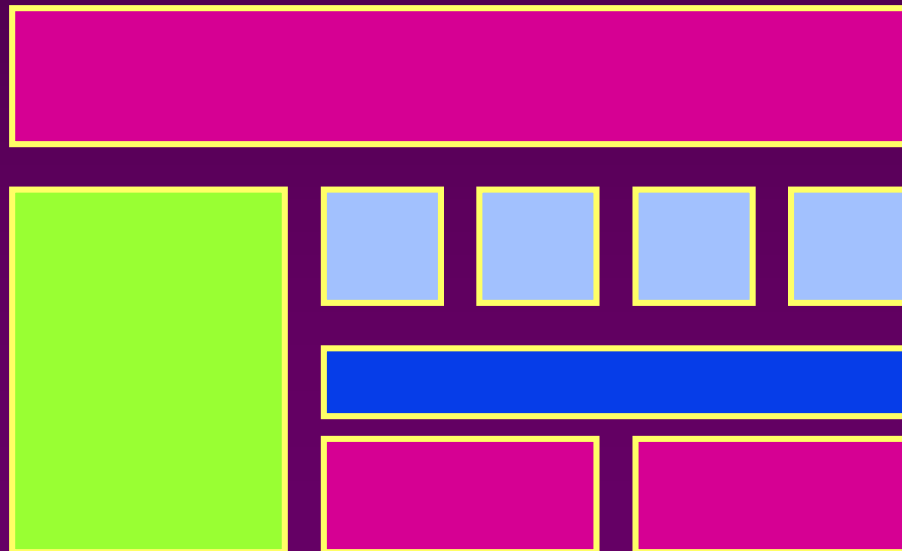


Structures

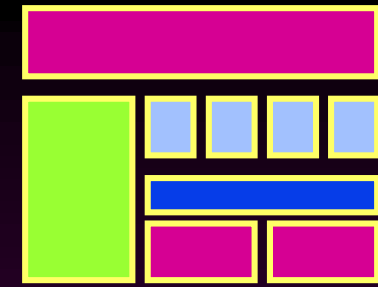


Pointers



Datatypes → int, float, char

Structures



- A **Structure** is a collection of related data items, possibly of different types. ← datatypes
- A structure type in C++ is called **struct**.
- A **struct** is **heterogeneous** in that it can be composed of data of different types.
- In contrast, **array** is **homogeneous** since it can contain only data of the same type.

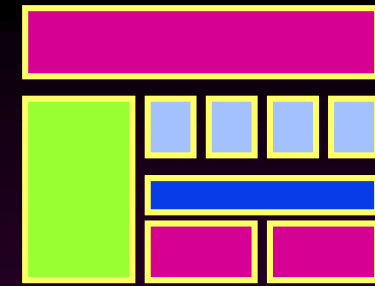
int
char
float

int arr[7];

char arr[7];

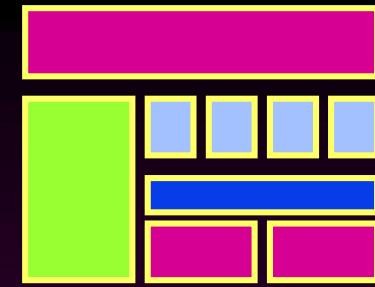
float arr[7];
100

Structures



- Structures hold data that belong **together**.
- Examples:
 - Student record: student id, name, major, gender, start year, ...
 - char* → gender
 - int* → student id
 - string, char, array* → name, major
 - Bank account: account number, name, currency, balance, ...
 - Address book: name, address, telephone number, ...
- In database applications, structures are called records.

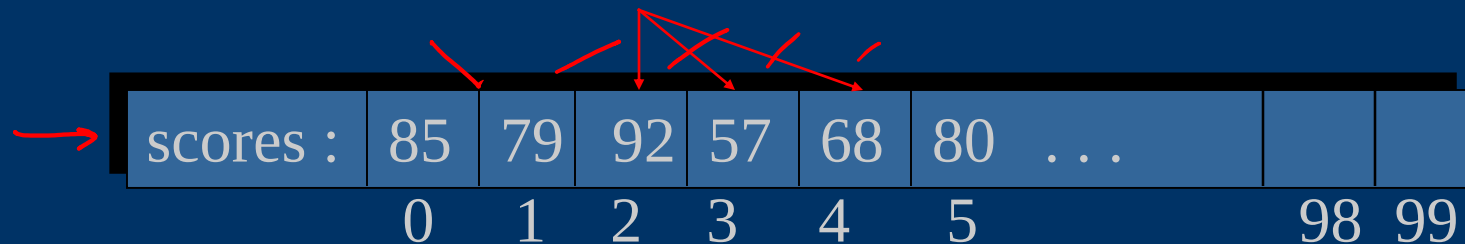
Structures



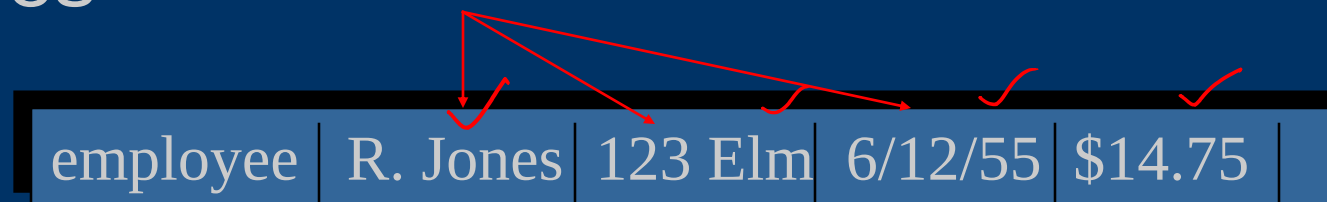
- Individual components of a struct type are called members (or fields).
- Members can be of different types (simple, array or struct).
- A struct is named as a whole while individual members are named using field identifiers.
- Complex data structures can be formed by defining arrays of structs.

Structures

- ◆ Recall that elements of arrays must all be of the same type



- ◆ In some situations, we wish to group elements of different types



Structures

- ◆ RECORDS are used to group related components of different types
- ◆ Components of the record are called fields

- ◆ In C++

employee	Name	123	6/12/95	PKR 40K
----------	------	-----	---------	---------

- record called a struct (structure)
- fields called members

type def

Structures

◆ C++ struct

- structured data type
- fixed number of components
- elements accessed by name, not by index
- components may be of different types

⇒ struct studentRecord

{
 1 → int student_ID;
 2 → string firstName;
 3 → char gender;
 };

type Name;
 studentRecord st;
 ↓
 "." dot
 3 member

```
struct part_struct {
  char descrip [31], part_num [11];
  float unit_price;
  int qty; };
```

Declaring struct Variables

◆ Given

```
struct part_struct {  
    char descrip [31], part_num [11];  
    float unit_price;  
    int qty; };
```

◆ Declare :

Use struct name as a type.

int newPrice, oldPrice; ←

part_struct new_part, old_part;

Variables

descrip part_num unit_price qty

Accessing Components

- ◆ Use the name of the record
the name of the member separated by a dot .

o $ld_part = 3$

$cout \ll old_part \cdot qty;$

```
old_part.qty = 5;
```

```
cout << new_part.descrip;
```

- ◆ The dot is called the member selector

// we want to enter 2 students data

cmsID
Name
gender
Hssc%

ID	Name	gender	%	Disp
✓	✓	✓	✓	✓
✓	✓	✓	✓	✓

```
int cmsID [2];
string Name[2];
char gender [2];
float HsscPer [2];
```

cin >> cmsID;

cin >> Name;

Structure ⇒ data

```
struct StudentData {
    int cmsID;
    string Name;
    char gender;
    float HsscPer;
};
```

StudentData data ;

cin >> data . cmsID ;

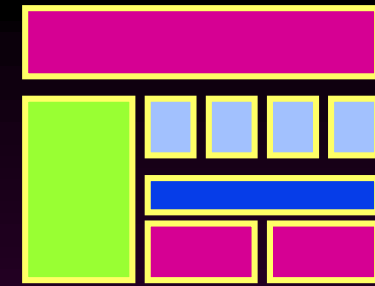
cin >> data . Name ;

function → calculator

add sub Div mu

cin >> data??

struct basics



- Definition of a structure:

```
struct <struct-type>{  
    <type> <identifier_list>;  
    <type> <identifier_list>;  
    ...  
} ;
```

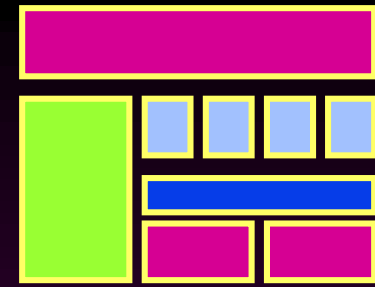
Each identifier defines a member of the structure.

- Example:

```
struct Date {  
    int day;  
    int month;  
    int year;  
} ;
```

The “Date” structure has 3 members, day, month & year.

struct examples



- Example:

```
struct StudentInfo{  
    int Id;  
    int age;  
    char Gender;  
    double CGA;  
};
```

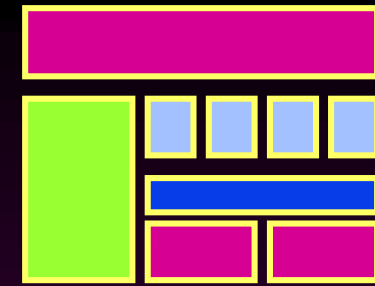
The “StudentInfo”
structure has 4 members
of different types.

- Example:

```
struct StudentGrade{  
    char Name[15];  
    char Course[9];  
    int Lab[5];  
    int Homework[3];  
    int Exam[2];  
};
```

The “StudentGrade”
structure has 5
members of
different array types.

struct examples



- Example:

```
⇒ struct BankAccount{  
    _char Name[15];  
    _int AccountNo[10];  
    double balance;  
    Date Birthday;  
};
```

m → *d* → *structure*

aslam.Birthday.m, year

aslam.Birthday.day;

aslam.Birthday.year;

The “BankAccount” structure has simple, array and structure types as members.

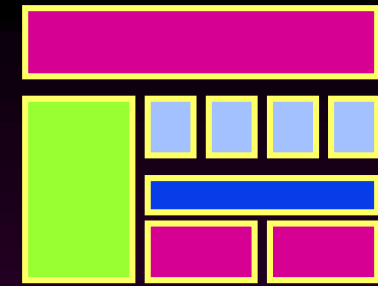
- Example:

```
struct StudentRecord{  
    char Name[15];  
    int Id;  
    char Dept[5];  
    char Gender;  
};
```

The “StudentRecord” structure has 4 members.

HL-7

struct basics

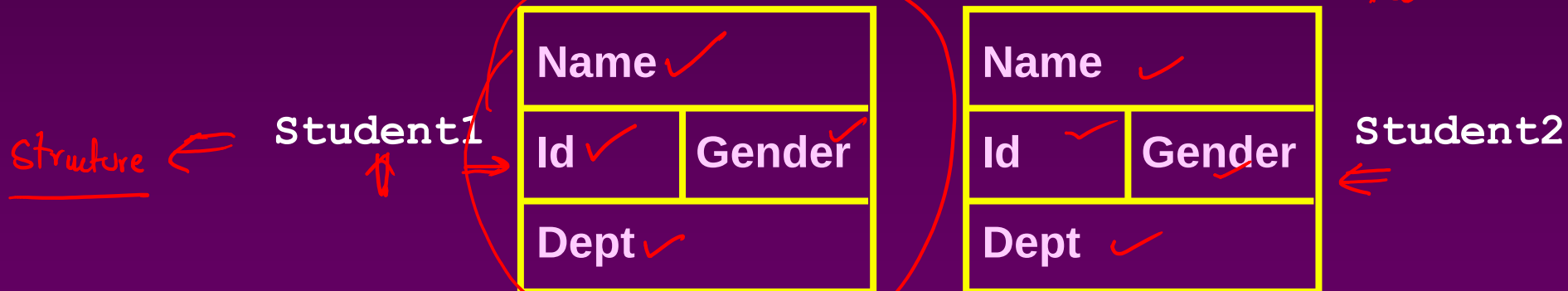
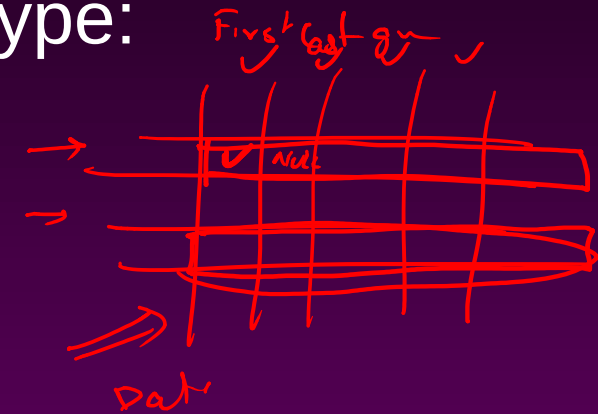


- Declaration of a variable of struct type:

```
<struct-type> <identifier_list>;
```

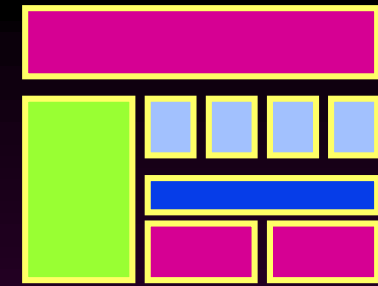
- Example:

```
StudentRecord Student1, Student2;
```



Student1 and Student2 are variables of StudentRecord type.

struct basics

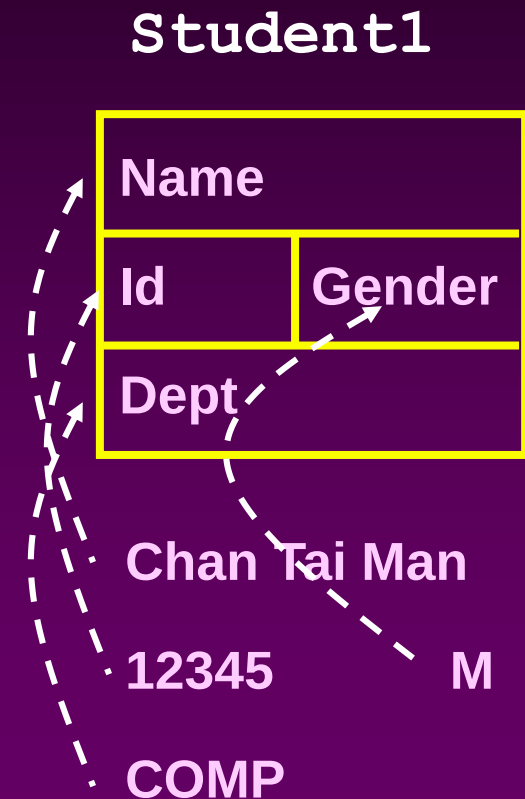


- The members of a **struct** type variable are accessed with the dot (.) operator:

`<struct-variable>.<member_name>;`

- Example:

```
strcpy(Student1.Name, "Chan Tai Man");
Student1.Id = 12345;
strcpy(Student1.Dept, "COMP");
Student1.gender = 'M';
cout << "The student is ";
switch (Student1.gender) {
    case 'F': cout << "Ms. "; break;
    case 'M': cout << "Mr. "; break;
}
cout << Student1.Name << endl;
```



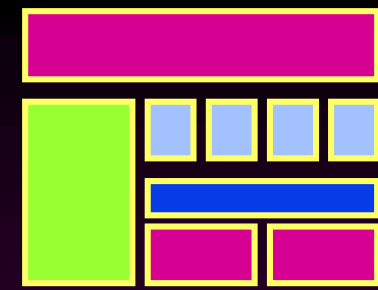
```
#include <string.h>
```

```
struct StudentRecord {  
    char Name[22];  
    int Id;  
    char Dept[22];  
    char gender;  
};
```

```
int main() {  
    StudentRecord Student1[10];  
    strcpy(Student1.Name, "Chan Tai Man");  
    Student1.Id = 12345;  
    strcpy(Student1.Dept, "COMP");  
    Student1.gender = 'M';  
  
    cout << "The student is ";  
    switch (Student1.gender){  
        case 'F': cout << "Ms. "; break;  
        case 'M': cout << "Mr. "; break;  
    }  
    cout << Student1.Name << endl;  
    return 0;  
}
```

```
MS-DOS struct  
Auto  
The student is Mr. Chan Tai Man  
Press any key to continue_
```

struct-to-struct assignment

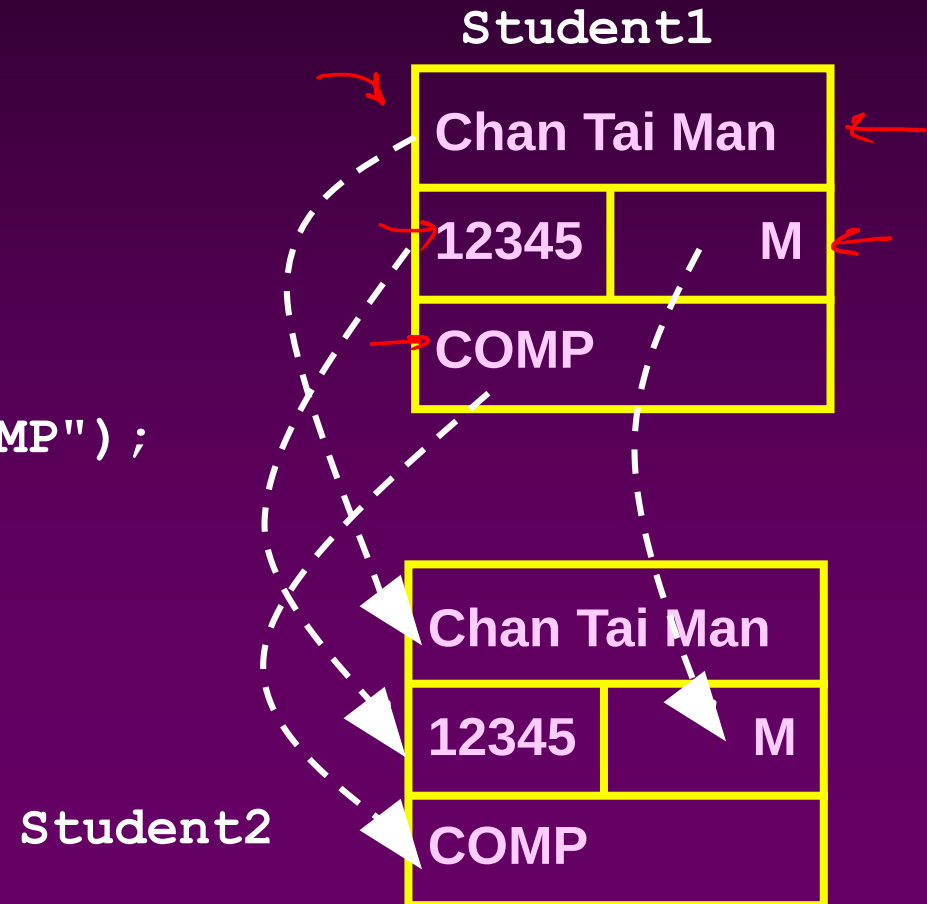


- The values contained in one struct type variable can be assigned to another variable of the same struct type.

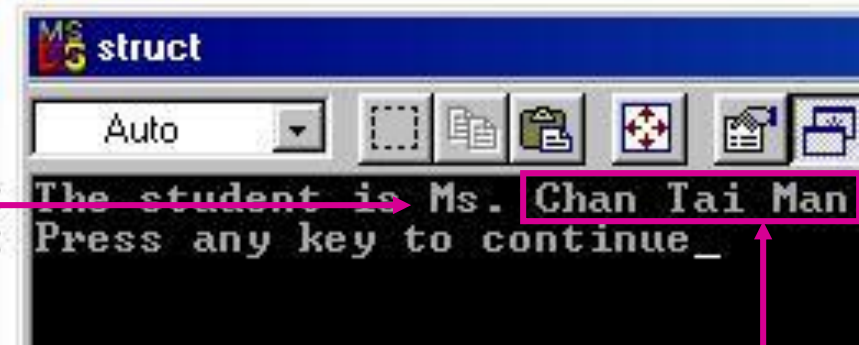
- Example:

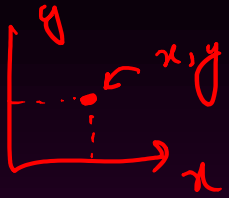
array = 0
⇒ Student1
Student[i].Name;

```
strcpy(Student1.Name, "Chan Tai Man");  
Student1.Id = 12345;  
strcpy(Student1.Dept, "COMP");  
Student1.gender = 'M';  
  
Student2 = Student1;
```



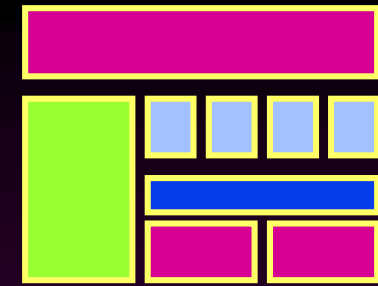
```
struct StudentRecord {  
    char Name[22];  
    int Id;  
    char Dept[22];  
    char gender;  
};  
  
int main() {  
    StudentRecord Student1, Student2;  
  
    strcpy(Student1.Name, "Chan Tai Man");  
    Student1.Id = 12345;  
    strcpy(Student1.Dept, "COMP");  
    Student1.gender = 'M';  
  
    Student2 = Student1;  
    Student2.gender = 'F';  
  
    cout << "The student is ";  
    switch (Student2.gender) {  
        case 'F': cout << "Ms. "; break;  
        case 'M': cout << "Mr. "; break;  
    }  
    cout << Student2.Name << endl;  
    return 0;  
}
```





point x_1 ; $x_1 = x$
 $x_1 = y$

Nested structures



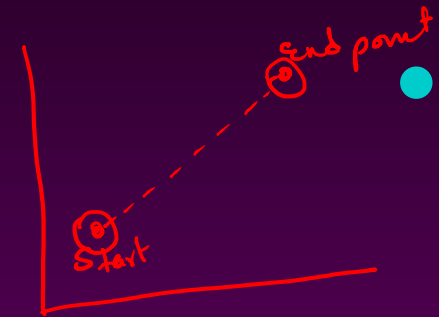
- We can nest structures inside structures.

- Examples:

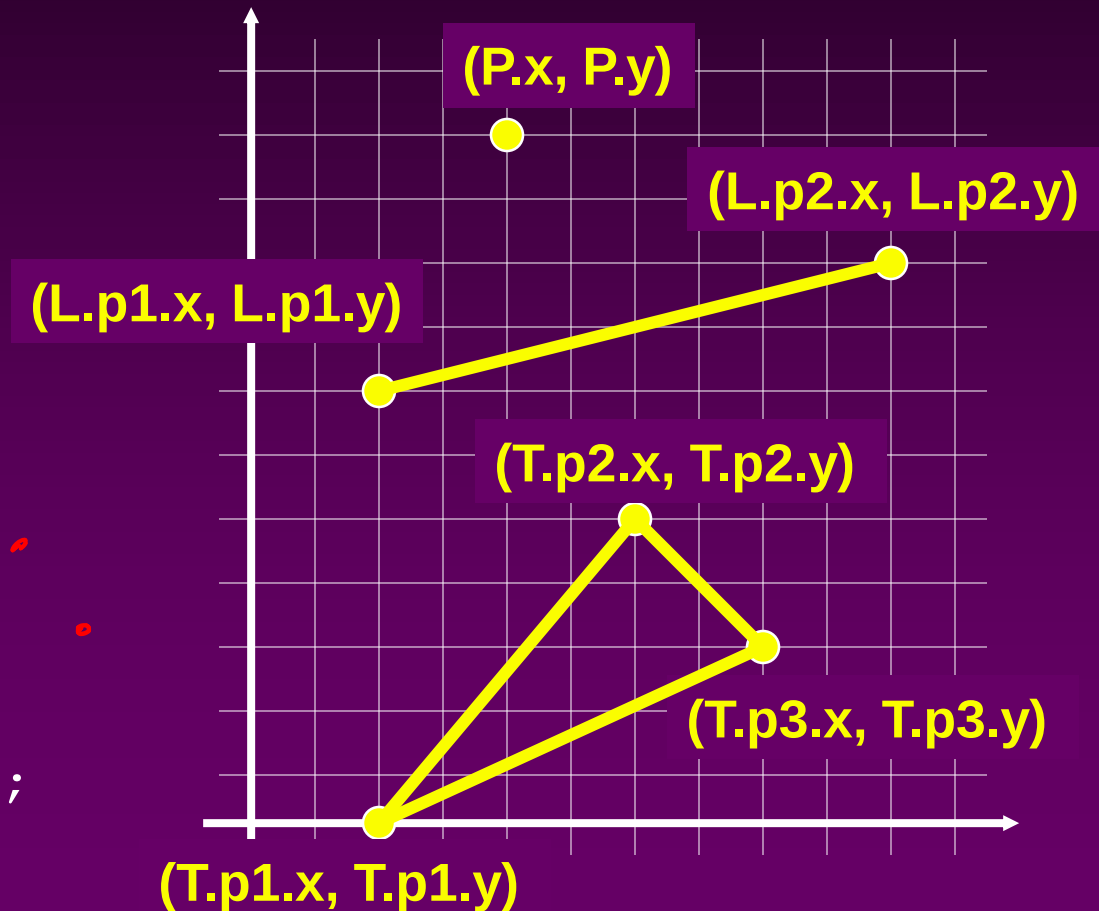
```
struct point{
    double x, y;
};
point P;
```

```
→ struct line{
    point p1, p2;
};
line L;
```

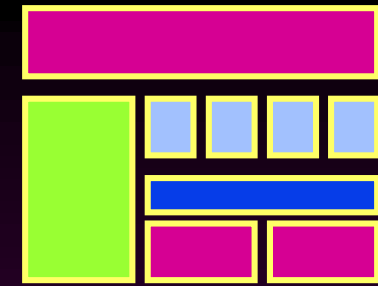
```
struct triangle{
    point p1, p2, p3;
};
triangle T;
```



line $L = 1$;
 $L1.p1 = x$;
 $L1.p1 = y$;
 $L1.p2 = x$;
 $L1.p2 = y$;

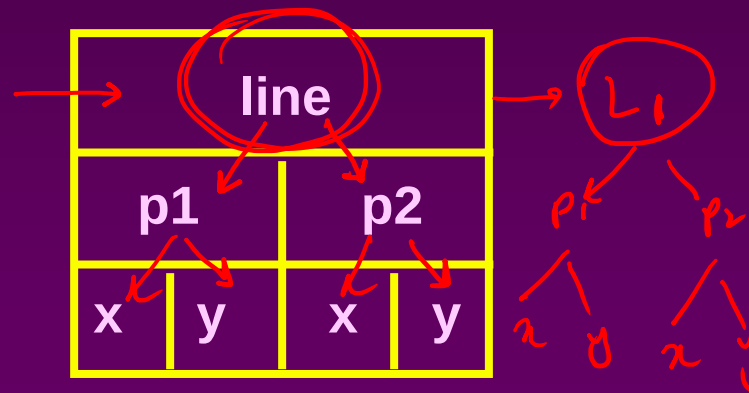


Nested structures



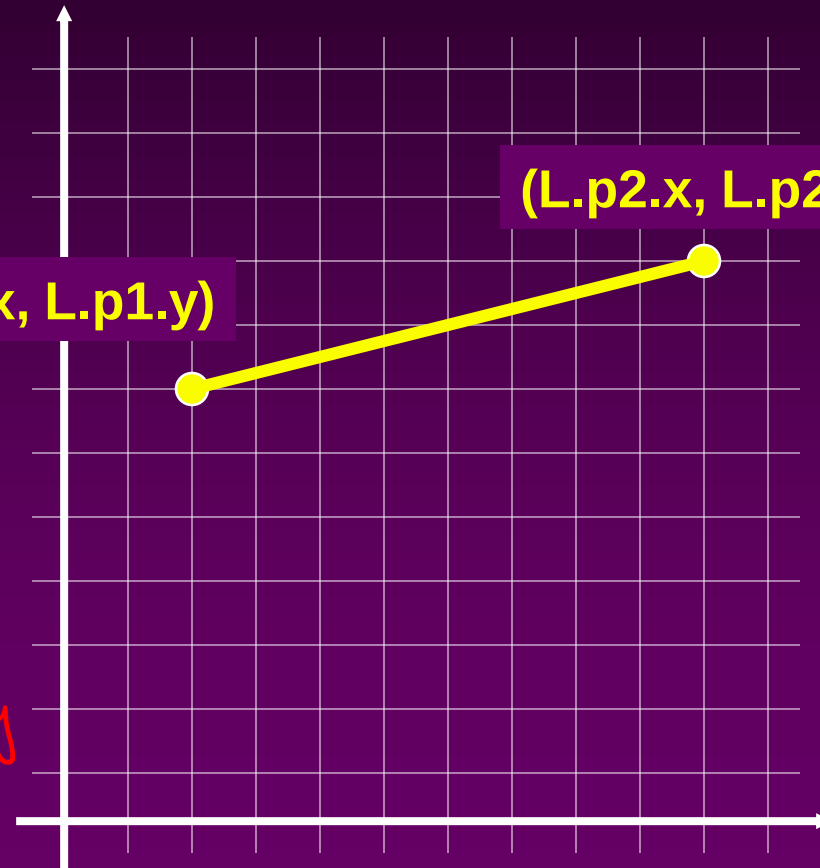
- We can nest structures inside structures.

```
• struct line{  
    point p1, p2;  
};  
line L;
```

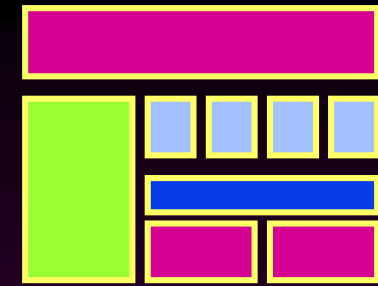


(L.p1.x, L.p1.y)

(L.p2.x, L.p2.y)



Nested structures



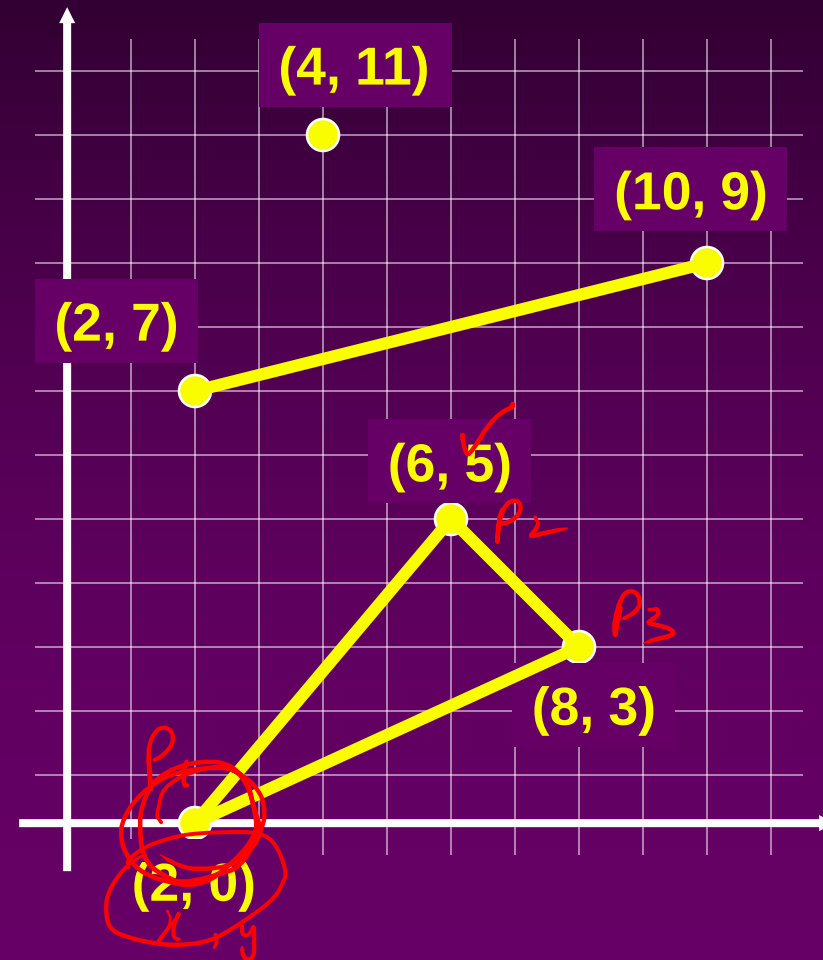
- Assign values to the variables `p`, `L`, and `T` using the picture:

```
struct point
{
    double x,y;
}

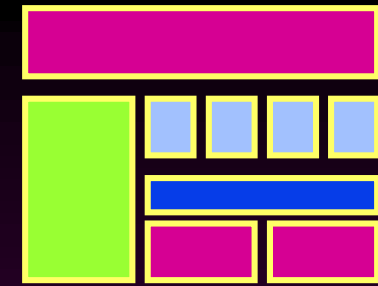
struct Triangle
{
    point p1, p2, p3;
}

point P;
line L;
triangle T;
```

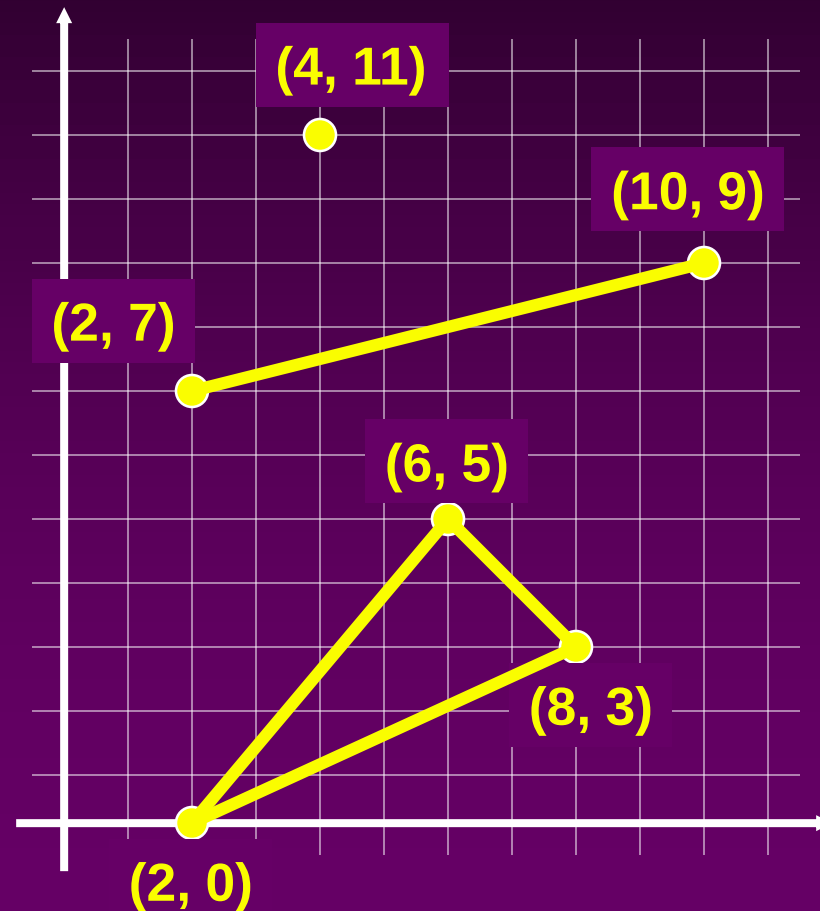
```
cin >> T.p1.x; // 2
cin >> T.p1.y; // 0
cin >> T.p2.x; // 6
cin >> T.p2.y; // 5
cin >> T.p3.x; // 8
cin >> T.p3.y; // 3
```



Nested structures



```
point P;  
line L;  
triangle T;  
  
P.x = 4;  
P.y = 11;  
  
L.p1.x = 2;  
L.p1.y = 7;  
L.p2.x = 10;  
L.p2.y = 9;  
  
T.p1.x = 2;  
T.p1.y = 0;  
T.p2.x = 6;  
T.p2.y = 5;  
T.p3.x = 8;  
T.p3.y = 3;
```



Structure as Functions

```
#include <iostream>
using namespace std;
struct Person
{
    char name[50];
    int age;
    float salary;
};
void displayData(Person); // Function declaration
int main()
{
    Person p;
    cout << "Enter Full name: ";
    cin.get(p.name, 50);
    cout << "Enter age: ";
    cin >> p.age;
    cout << "Enter salary: ";
    cin >> p.salary;
```

```
// Function call with structure variable as an argument
displayData(p);
return 0;
}
void displayData(Person p)
{
    cout << "\nDisplaying Information." << endl;
    cout << "Name: " << p.name << endl;
    cout << "Age: " << p.age << endl;
    cout << "Salary: " << p.salary;
}
```

Structure as Functions

```
#include <iostream>
using namespace std;
struct Person {
    char name[50];
    int age;
    float salary;
};
Person getData(Person);
void displayData(Person);
int main()
{
    Person p;
    p = getData(p);
    displayData(p);
    return 0;
}
Person getData(Person p) {
    cout << "Enter Full name: ";
    cin.get(p.name, 50);
```

```
    cout << "Enter age: ";
    cin >> p.age;
    cout << "Enter salary: ";
    cin >> p.salary;
    return p;
}
void displayData(Person p)
{
    cout << "\nDisplaying Information." << endl;
    cout << "Name: " << p.name << endl;
    cout << "Age: " << p.age << endl;
    cout << "Salary: " << p.salary;
}
```